

Dependency Injection

Il **provider** è ciò che crea la dipendenza per conto del sistema di dependency injection di Angular.

In qualche modo, il sistema di dependency injection deve creare un'istanza del **CoursesService**.

Il modo per creare questa dipendenza è proprio il **provider**.

Il provider fornisce al sistema di dependency injection una funzione chiamata **factory function**, che serve a creare l'istanza della dipendenza.

Il sistema di dependency injection di Angular chiamerà poi questa factory function per generare la dipendenza.

Di solito specifichiamo tutto questo nel decoratore **@Injectable**:

```
@Injectable({
  providedIn: 'root'
})
```

Ma possiamo anche provare a scrivere manualmente un **provider**, ad esempio direttamente all'interno di un componente:

```
function coursesServiceProvider(http: HttpClient): CoursesService {
  return new CoursesService(http)
}
```

You, 1 second ago | 4 authors (Your Name and others)

```
@Component({
  selector: 'course-card',
  templateUrl: './course-card.component.html',
  styleUrls: ['./course-card.component.css'],
  standalone: false
})
```

Come puoi vedere, il **provider** è una semplice funzione che restituisce una nuova istanza del servizio.

Ora, se vogliamo passare questa funzione al componente, dobbiamo definire la proprietà **providers** nel decoratore del componente, passando un oggetto di configurazione come questo:

```
function coursesServiceProvider(http: HttpClient): CoursesService {
  return new CoursesService(http)
}
```

```
export const COURSES_SERVICE = new InjectionToken<CoursesService>('COURSES_SERVICE')
```

You, 15 seconds ago | 3 authors (JHades and others)

```
@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css'],
  providers: [
    {
      provide: COURSES_SERVICE,
      useFactory: coursesServiceProvider,
      deps: [HttpClient]
    }
  ]
})
```

Ogni dipendenza nel sistema di **dependency injection** deve avere un nome univoco.

Per questo motivo creiamo un **InjectionToken**, che funziona come un identificatore unico per la dipendenza.

Dopo aver definito la **factory function**, informiamo Angular su come creare il servizio **CourseService**, specificando due elementi:

- il **token** univoco per la dipendenza;
- la **factory function** che Angular chiamerà per ottenere l'istanza del servizio.

Infine, dobbiamo specificare nella proprietà **deps** le dipendenze del servizio, cioè altri servizi che la **factory function** richiede per funzionare.

Ora siamo pronti per iniettare il servizio nel **costruttore**:

```
constructor(@Inject(COURSES_SERVICE) private coursesService: CoursesService) {
```

Dobbiamo specificare l'**injection token** all'interno del decoratore **@Inject**.

Questo esempio era a scopo teorico, perché Angular supporta direttamente come **injection token** sia i **nomi delle classi** che i **costruttori delle classi** come factory function.

Quindi, in pratica, non è necessario definire manualmente il token e la factory function nella maggior parte dei casi.

```
providers: [  
  {  
    provide: CoursesService,  
    useClass: CoursesService  
  }  
]
```

In modo più compatto, possiamo semplicemente passare il **nome della classe** all'array **providers**:

```
providers: [  
  CoursesService  
]
```

Se forniamo i servizi in questo modo, creeremo un'istanza per il **componente** e per tutti i **componenti figli**.

Se invece forniamo un **CoursesService** in un componente figlio, verrà creata un'altra istanza separata.

Quando iniettiamo un servizio nel costruttore, Angular cercherà un provider nell'ambito del **componente corrente**. Se non trova alcun provider, cercherà nei **componenti genitori**, risalendo la gerarchia.

Questo meccanismo si chiama **Hierarchical Dependency Injection**.

Se vogliamo essere sicuri di avere una **istanza singleton** (cioè un'unica istanza condivisa in tutta l'app), possiamo usare:

```
@Injectable({
  providedIn: 'root'
})
export class CoursesService {
```

Per farlo, forniamo il servizio direttamente nella classe del servizio stesso utilizzando il decoratore **providedIn**, questo approccio è chiamato **Tree-Shakeable Syntax**.

In questo modo Angular può ottimizzare il bundle finale eliminando i servizi non utilizzati.

Possiamo anche iniettare servizi nelle **direttive**.

Se vogliamo essere sicuri che l'istanza del servizio venga presa dal **componente host** della direttiva (cioè il componente su cui è applicata la direttiva), possiamo usare il decoratore **@Host** nel costruttore della direttiva:

```
constructor(@Host() private coursesService: CoursesService) {}
```

Invece, Angular cercherà di rilevare i cambiamenti nei dati **Input** del componente.

Nota che, se viene attivato un **gestore di eventi**, anche con la modalità **OnPush**, il rilevamento delle modifiche (**change detection**) verrà comunque attivato, proprio come accade nella modalità di default.

Con la modalità **OnPush**, se sappiamo che gli **Input** di un determinato ramo dell'albero dei componenti non sono cambiati, possiamo **saltare l'intero ramo**.

Ad esempio, se il componente **CourseCard** avesse dei componenti figli, questi non verrebbero controllati dal change detection se il **CourseCard** stesso non ha ricevuto nuovi valori nei suoi Input. In questo modo, un intero ramo dell'albero dei componenti viene bypassato, migliorando le prestazioni.

Inoltre, nel caso di dati Input come gli oggetti JavaScript (ad esempio un oggetto **course**), il controllo avviene per **riferimento**: viene verificato se il riferimento all'oggetto è cambiato.

Il change detection con **OnPush** controlla anche eventuali **stream osservabili** registrati nel template usando la **async pipe**, e verifica se un nuovo valore è stato emesso dall'osservabile.

Quindi, con **OnPush**, ci sono **due modi principali** per aggiornare un componente:

1. Tramite **proprietà Input** del componente;
2. Tramite **stream osservabili** collegati con l'**async pipe**.

Entrambi i metodi faranno sì che il componente venga nuovamente renderizzato.

Se abbiamo una proprietà **Input** che rimane costante per tutta la durata dell'applicazione, ad esempio:

```
[type]='BEGINNER'
```

Invece di passare il valore come proprietà **Input**, possiamo usare il decoratore **@Attribute**, che permette di leggere direttamente un **attributo statico** dal template.

Questo approccio migliora le **prestazioni dell'applicazione**, perché gli attributi non sono soggetti a change detection: vengono letti una volta sola, all'inizializzazione della direttiva o del componente.

```
type="BEGINNER"
```

E nel costruttore del componente figlio (o della direttiva figlia), scriveremo:

```
constructor( @Attribute('type') private type: string) { }
```

Questo si chiama **One-Time Binding**, ovvero **binding a una sola lettura**.

Con il decoratore **@Attribute**, il valore dell'attributo viene letto **solo una volta**, al momento della creazione del componente o della direttiva, e **non viene più aggiornato** anche se l'attributo cambia successivamente nel template.

Questo approccio è utile quando sappiamo che il valore dell'attributo è **statico** e non deve partecipare al ciclo di change detection di Angular, migliorando così le prestazioni.